

SQL und NoSQL

Was ist NoSQL?

NoSQL steht für "Not only SQL". Gemeint sind alle Datenbanken, die Daten nicht in Tabellen mit festen Spalten und Fremdschlüsseln speichern. Es ist also kein einzelnes Produkt, sondern ein Sammelbegriff für verschiedene Bauarten.

NoSQL-Datenbanken sind ab etwa 2009 bekannt geworden, als große Webdienste an Grenzen stießen:

- Datenmenge: Milliarden von Datensätzen passen nicht mehr auf einen einzelnen Server.
- Zugriffe: Hunderttausende Anfragen pro Sekunde.
- Wechselnde Daten: Jeder Datensatz sieht etwas anders aus (zum Beispiel Produkte mit völlig unterschiedlichen Eigenschaften).

Relationale Datenbanken wachsen klassisch vertikal: ein größerer Server. NoSQL-Datenbanken sind meist von Anfang an für horizontales Wachstum gebaut: einfach weitere Server dazustellen und die Daten verteilen.

Die vier Arten von NoSQL

Key-Value-Datenbank (Schlüssel und Wert)

Die einfachste Form: wie eine riesige HashMap in Java. Zu einem Schlüssel gehört ein Wert. Die Datenbank weiß nicht, was im Wert steht.

```
SET    warenkorb:4711 "{artikel: 12, menge: 2}"
GET    warenkorb:4711
EXPIRE warenkorb:4711 3600      -- nach einer Stunde automatisch löschen
```

Produkte: Redis, Valkey, Amazon DynamoDB.

Typisch für: Sitzungen (Wer ist gerade angemeldet?), Warenkörbe, Zwischenspeicher (Cache), Zähler.

Dokumentdatenbank

Speichert ganze Dokumente, meist im JSON-Format. Ein Dokument kann verschachtelt sein und Listen enthalten. Zwei Dokumente in derselben Sammlung (Collection) dürfen unterschiedlich aufgebaut sein.

```
{
  "_id": "B-2026-0815",
  "datum": "2026-10-08",
  "kunde": { "name": "Meier", "ort": "Unna" },
  "positionen": [
    { "artikel": "USB-Kabel", "menge": 2, "preis": 4.99 },
    { "artikel": "Maus", "menge": 1, "preis": 19.90 }
  ]
}
```

```
db.bestellungen.find({ "kunde.name": "Meier" })
```

Produkte: MongoDB, CouchDB.

Typisch für: Produktkataloge mit unterschiedlichen Eigenschaften, Inhalte (Artikel, Kommentare), Profile, Daten aus Apps.

Spaltenfamilien-Datenbank (Wide Column)

Speichert Zeilen mit einem Schlüssel, aber jede Zeile darf andere und sehr viele Spalten haben. Die Daten werden über viele Server verteilt und sind auf extrem viele Schreibzugriffe ausgelegt.

```
Zeilenschlüssel: sensor-17
2026-10-08T10:00 = 21.4
2026-10-08T10:01 = 21.5
2026-10-08T10:02 = 21.7 ... beliebig viele weitere Spalten
```

Produkte: Apache Cassandra, ScyllaDB, HBase.

Typisch für: Messwerte von Sensoren, Protokolldaten (Logs), Klickverläufe, alles mit sehr vielen Schreibvorgängen.

Graphdatenbank

Speichert Knoten (Dinge) und Kanten (Beziehungen). Die Beziehung selbst ist ein gespeichertes Objekt und muss nicht erst über einen JOIN zusammengesetzt werden.

(Anna) -[:KAUFTE]-> (Maus) <-[:KAUFTE]- (Ben) -[:KAUFTE]-> (Mauspad)

```
MATCH (k:Kunde {name: "Anna"})-[:KAUFTE]->(a)<-[:KAUFTE]-(andere)-[:KAUFTE]->(tipp)
RETURN tipp
```

Die Abfrage liefert: "Kunden, die das gekauft haben, kauften auch ..." (hier: Mauspad).

Produkte: Neo4j (Abfragesprache Cypher), Amazon Neptune.

Typisch für: Empfehlungen, soziale Netzwerke, Routenplanung, Betrugserkennung.

Dieselben Daten: relational und als Dokument

Eine Bestellung mit zwei Positionen.

Relational (MariaDB)	Dokument (MongoDB)
tbl_kunde: 1 Zeile tbl_bestellung: 1 Zeile mit kunde_id tbl_bestellposition: 2 Zeilen mit bestellung_id tbl_artikel: 2 Zeilen Zum Anzeigen werden die Tabellen mit JOIN zusammengesetzt.	Ein einziges Dokument. Kunde und Positionen stecken darin. Zum Anzeigen wird genau ein Dokument gelesen. Kein JOIN.
Ändert der Kunde seinen Namen, wird eine Zeile geändert.	Steht der Kunde in 50 Bestellungen, steht sein Name 50 Mal da. Man muss entscheiden, ob das gewollt ist.

Relational: Daten aufteilen, beim Lesen zusammensetzen (normalisieren).
Dokument: Daten so speichern, wie sie gelesen werden (einbetten).

Die Unterschiede im Überblick

Merkmal	SQL (relational)	NoSQL
Datenmodell	Tabellen mit Zeilen und Spalten	Schlüssel-Wert, Dokument, Spaltenfamilie oder Graph
Schema	fest: vorher mit CREATE TABLE festgelegt	flexibel: jeder Datensatz darf anders aussehen
Beziehungen	Fremdschlüssel, JOIN, von der Datenbank geprüft	eingebettet oder als Verweis. Prüfung meist im Programm (Ausnahme: Graph)
Abfragesprache	SQL, genormt, in allen Produkten fast gleich	je Produkt eigene Sprache oder Schnittstelle
Konsistenz	ACID: sofort und immer konsistent	oft BASE: irgendwann konsistent (siehe Abschnitt 5)
Wachstum	vor allem vertikal (größerer Server)	horizontal (mehr Server)
Regeln in der Datenbank	NOT NULL, UNIQUE, CHECK, Löschregeln	wenige oder keine, das Programm muss aufpassen
Bekannte Produkte	MariaDB, MySQL, PostgreSQL, Oracle, Microsoft SQL Server	MongoDB, Redis, Cassandra, Neo4j

ACID und BASE

Relationale Datenbanken arbeiten nach ACID (Atomarität, Konsistenz, Isolation, Dauerhaftigkeit).

Eine bestätigte Änderung ist sofort für alle sichtbar und gültig.

Viele verteilte NoSQL-Datenbanken arbeiten nach BASE:

Buchstaben	Bedeutung
BA: Basically Available	Die Datenbank antwortet fast immer, auch wenn einzelne Server ausfallen.
S: Soft State	Der Zustand kann sich ändern, während die Server sich abgleichen.
E: Eventually Consistent	Nach kurzer Zeit haben alle Server denselben Stand. Bis dahin kann ein Server noch alte Daten liefern.

Beispiel: Sie liken einen Beitrag. Ein Freund sieht die Zahl der Likes zwei Sekunden später. Das ist egal. Bei einem Kontostand oder einem Lagerbestand wäre das nicht egal.

Das CAP-Theorem

Ein verteiltes System kann bei einer Netzstörung zwischen den Servern nicht gleichzeitig alles garantieren:

- **C**onsistency: Alle sehen sofort denselben Stand.
- **A**vailability: Jede Anfrage bekommt eine Antwort.
- **P**artition Tolerance: Das System läuft weiter, auch wenn die Verbindung zwischen Servern abreißt.

Netzstörungen kommen vor, also muss P immer gelten. Dann muss man wählen: lieber **C** (im Zweifel nicht antworten, Beispiel Bank) oder lieber **A** (im Zweifel einen alten Stand liefern, Beispiel soziales Netzwerk).

Vor- und Nachteile

Relationale Datenbanken (SQL)

Vorteile	Nachteile
• Datenbank prüft selbst: Schlüssel, Pflichtfelder, Löschregeln • ACID-Transaktionen, sicher für Geld und Belege • keine doppelten Daten durch Normalisierung • SQL ist genormt und seit Jahrzehnten bekannt • beliebige Auswertungen mit JOIN und GROUP BY möglich	• Schema muss vorher feststehen, Änderungen brauchen ALTER TABLE • Wachstum über viele Server ist aufwändig • viele JOINS über große Tabellen können langsam werden • unterschiedlich aufgebaute Daten passen schlecht in feste Spalten

NoSQL-Datenbanken

Vorteile	Nachteile
• flexibles Schema, neue Felder ohne Umbau • wächst einfach über viele Server • sehr schnell für die Zugriffe, für die das Modell gebaut ist • Daten liegen oft so vor, wie das Programm sie braucht (JSON)	• Datenbank prüft wenig, Fehler landen unbemerkt in den Daten • oft nur "irgendwann konsistent" • doppelte Daten müssen selbst aktuell gehalten werden • keine einheitliche Abfragesprache • Auswertungen über viele Datensätze und Beziehungen sind schwieriger

Wann nimmt man was?

Aufgabe	Passende Datenbank	Warum
Bestellungen, Rechnungen, Zahlungen	relational	Geld, Belege mit Aufbewahrungspflicht, Transaktionen nötig
Lagerbestand	relational	Der Bestand muss für alle sofort stimmen, sonst wird doppelt verkauft.
angemeldete Sitzungen, Warenkorb eines Besuchers	Key-Value	sehr schnell, darf nach Ablauf verschwinden
Produktkatalog mit sehr unterschiedlichen Produkten (Bücher, Kabel, Kleidung)	Dokument	jedes Produkt hat andere Eigenschaften
Kundenbewertungen und Kommentare	Dokument	viele, unterschiedlich lange Texte, kein Geld im Spiel
"Kunden kauften auch"	Graph	Beziehungen über mehrere Stufen sind der Kern der Frage
Klickverläufe, Server-Logs, Sensordaten	Spaltenfamilie	riesige Mengen, fast nur Schreiben, Auswertung später